📄 Functional Requirements Document (FRD)

Project: Enterprise Web UI Validation Engine – DemoQA Elements Module

1. Document Overview

This Functional Requirements Document (FRD) defines the expected behaviors, validation rules, and interaction constraints for the **Elements module of DemoQA Elements**.

The document serves as a **technical contract** between QA, Engineering, and Stakeholders, ensuring that:

- UI validation logic is clearly defined
- Negative and edge-case behaviors are testable
- Automation can reliably verify system responses

2. Feature: Text Box Validation

Requirement ID: REQ-TX-001

Title: Email Input Syntax Enforcement

Description

The system must validate the **Email input field (#userEmail)** using standard email format rules (aligned with RFC 5322 principles and browser-level validation).

Validation is enforced at the time of form submission.

Selectors

- Email Field: #userEmail
- Submit Button: #submit
- Output Container: #output

Behavior Specification

Condition	User Action	Expected UI State	Classification
Invalid Email Syntax (e.g., missing @, missing TLD)	Click Submit	Field enters invalid state (:invalid), border turns red (#dc3545), #output NOT rendered	VALIDATION_ERROR
Valid Email Syntax	Click Submit	#output container rendered with submitted data	SUCCESS

Additional Constraint

REQ-TX-002: Output Rendering Guard

- If any input field is invalid:
  - The #output container must NOT be rendered
  - Partial or inconsistent output must not appear

3. Feature: Web Tables Data Integrity

Requirement ID: REQ-WT-002

Title: Numeric Constraint & Modal Persistence

Description

The system must enforce numeric-only validation for the following fields within the **Registration Form modal**:

- Age (#age)
- Salary (#salary)

Validation occurs at submission time.

Selectors

- Add Button: #addNewRecordButton
- Age Field: #age
- Salary Field: #salary
- Submit Button: #submit
- Table Body: .rt-tbody

Behavior Specification

Condition	User Action	Expected UI State	Classification
Non-numeric input (e.g., abc, 12ab, @@@)	Click Submit	Submission blocked, modal remains open, no row added to .rt-tbody	VALIDATION_BLOCK
Valid numeric input	Click Submit	Modal closes, new row appended to .rt-tbody	SUCCESS

State Transition Rules

REQ-WT-003: Modal State Management

OPEN → INVALID INPUT → SUBMIT → REMAIN OPEN
OPEN → VALID INPUT → SUBMIT → CLOSE MODAL

4. Error Handling & State Transitions

Error Classification Model

Error Type	Description	Expected Outcome
VALIDATION_ERROR	Input format violation (e.g., invalid email)	Test should PASS (expected failure)
VALIDATION_BLOCK	Submission prevented due to invalid input	Test should PASS
INTERACTION_BLOCKED	Disabled element interaction (e.g., #noRadio)	No state change
SYSTEM_ERROR	JavaScript error, crash, DOM failure	Test should FAIL
ENVIRONMENTAL_BLOCK	Overlay or obstruction affecting interaction	Requires recovery strategy

Scenario Handling

Scenario	System Response	User Feedback
Mandatory field empty	Submission blocked	Browser-native tooltip or validation message
Disabled element interaction	Ignore pointer events	Cursor shows not-allowed, no UI change
Invalid input format	Field marked invalid	Visual indication (red border or invalid state)

5. Non-Functional Constraints (UI Context)

5.1 Lazy Loading & Visibility

- Elements must be:
  - Present in DOM
  - Visible (isVisible() == true)
- Automation must wait for readiness before interaction

5.2 Overlay Handling

The system must account for intermittent UI overlays such as:

- Floating advertisement banners
- Fixed-position elements

Expected Behavior

- Overlays must not cause:
  - Interaction failures
  - Application crashes
- Interaction recovery strategies may include:
  - Scroll into view
  - Re-attempt click after visibility adjustment

Constraint

Overlay conditions are considered **environmental**, not functional defects, unless they:

- Block interaction permanently
- Cause inconsistent UI state

🎯 Summary

This FRD defines:

✓ Exact validation rules
✓ DOM-level expectations
✓ Error classification model
✓ UI state transitions
✓ Automation constraints

If you want next, I can:

- Convert this into **BDD Gherkin scenarios (1:1 mapping)**
- Map each REQ → **automated test case IDs**
- Or generate **traceability matrix (FRD ↔ Tests ↔ Results)**